



An Axiomatic Concurrency Model

4

5



S



-

R  +

R.A. -



S



—



C



-

R  +

R.B. -



C



—





po

po







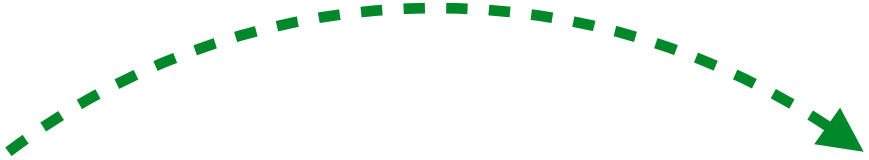
PO

po

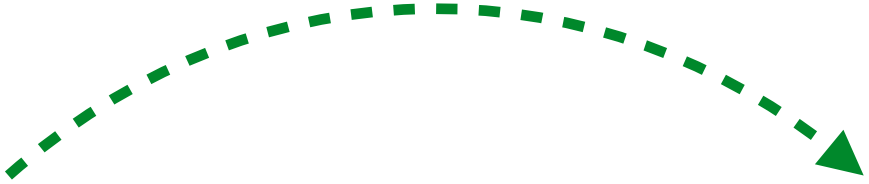


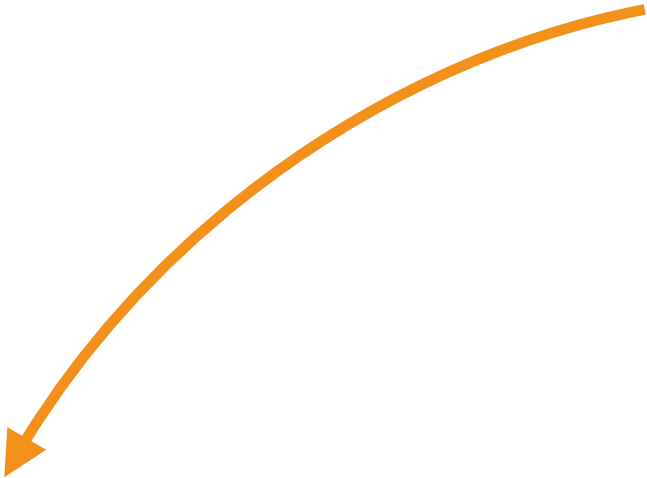
po

po

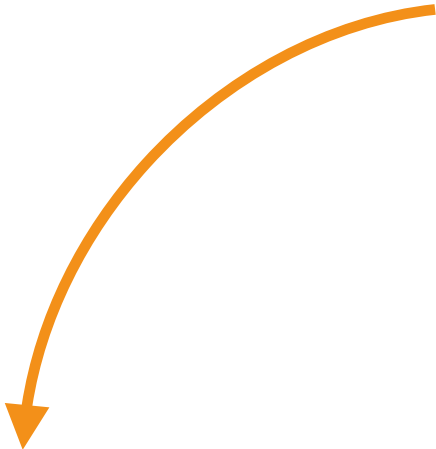




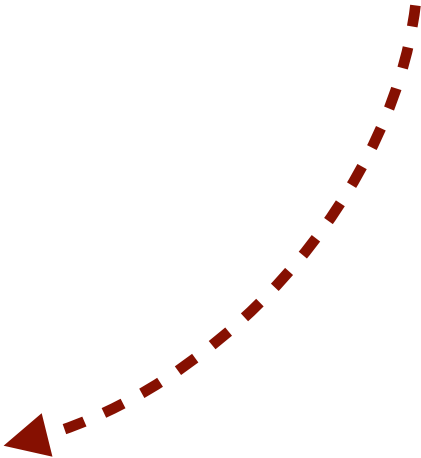


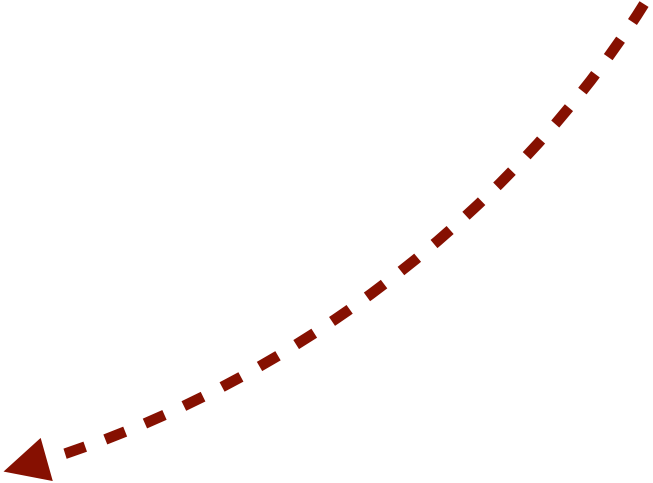


cosrc



collog





hb

h

b


```
update(src, +100)
```

```
update(src, -100)
```



Valid executions are acyclic in the transitive closure of the union of relations

```
def update(account: cown[Account], amount: int) {  
  when(account) { A  
    if(account.balance + amount > 0) {  
      account.balance += amount  
      val id = account.id()  
      when(log) { B log.record(id, amount) }  
    } } }  
}
```

R  +

R



-







po

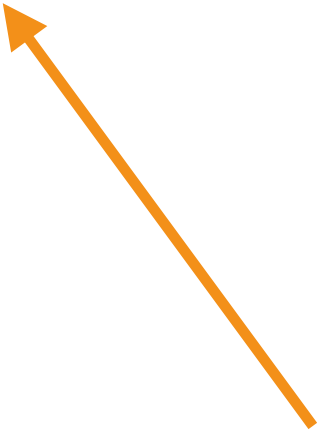
p

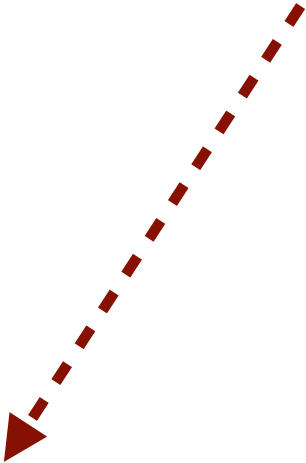
o



collog







h

b







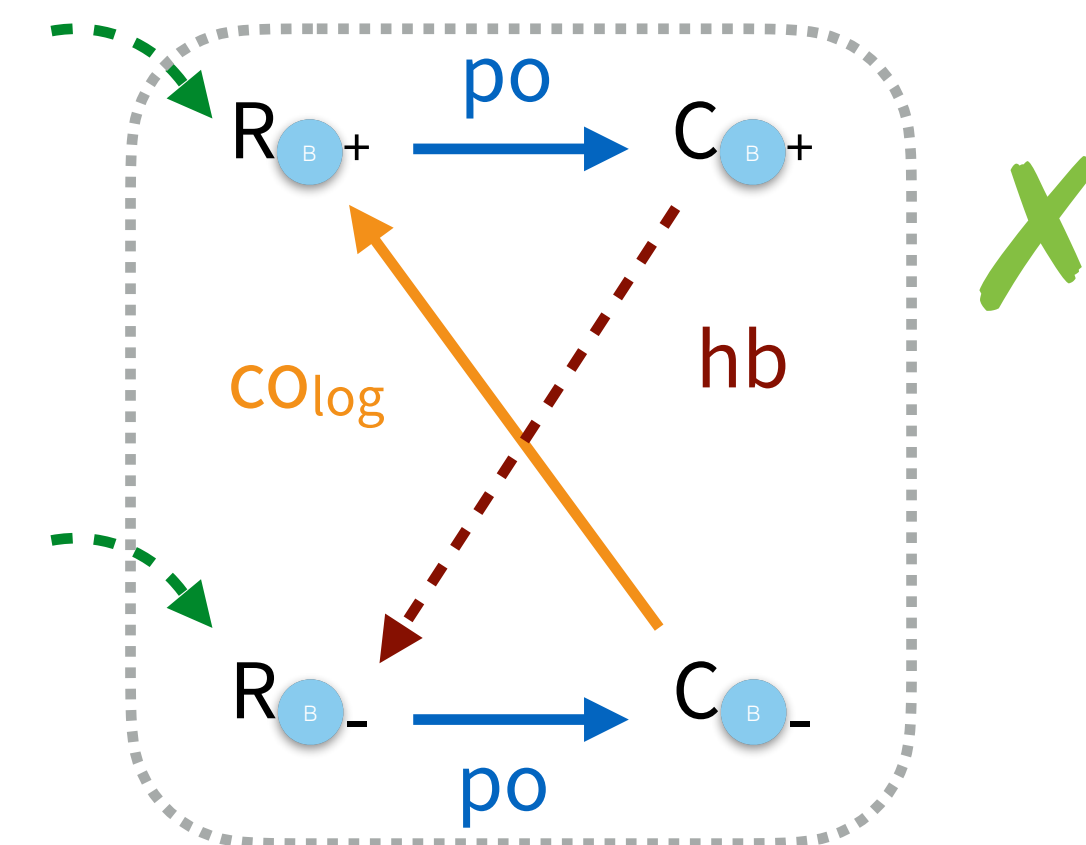
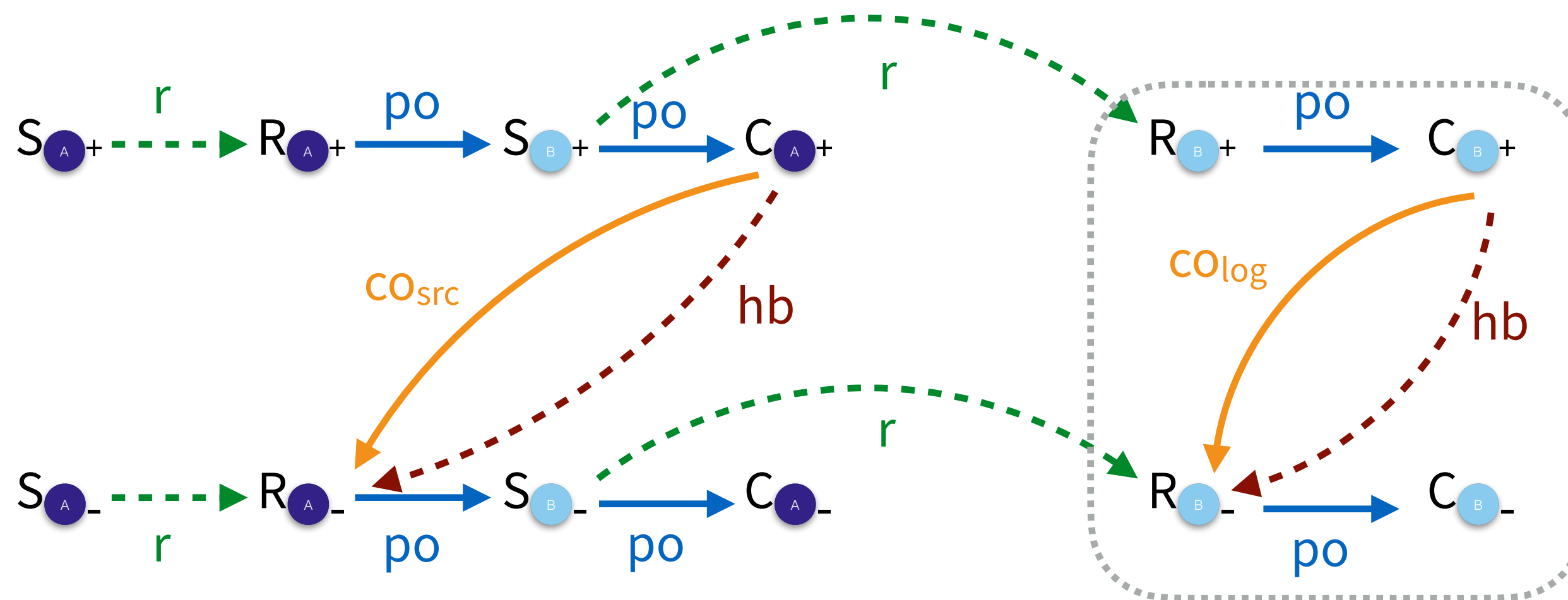
An Axiomatic Concurrency Model

```
def update(account: cown[Account], amount: int) {  
  when(account) { A  
    if(account.balance + amount > 0) {  
      account.balance += amount  
      val id = account.id()  
      when(log) { B log.record(id, amount) }  
    }  
  }  
}
```

update(src, +100)

update(src, -100)

Valid executions are acyclic in the transitive closure of the union of relations



What does this get us?

$$\begin{array}{c}
 \overline{\text{when}(\bar{c})\{s_1\}; s_2 \hookrightarrow s_2 \mid (\bar{c}, s_1)}^{\text{STEP}} \\
 \\
 \frac{s_1 \hookrightarrow s_2 \mid (\bar{c}_3, s_3) \quad j \text{ fresh}}{\langle \bar{b}_r[(i, \bar{c}_1, s_1)], \bar{b}_p \rangle / H \rightsquigarrow \langle \bar{b}_r[(i, \bar{c}_1, s_2)], \bar{b}_p : (j, \bar{c}_3, s_3) \rangle / H[i \vdash S_j]}^{\text{SPAWN}} \\
 \\
 \frac{C = \{c \mid c \in \bar{c}' \wedge (_, \bar{c}', _) \in \bar{b}_r \cup \bar{b}_{p1}\} \quad C \cap \bar{c} = \emptyset}{\langle \bar{b}_r, \bar{b}_{p1} : (i, \bar{c}, s) : \bar{b}_{p2} \rangle / H \rightsquigarrow \langle \bar{b}_r : (i, \bar{c}, s), \bar{b}_{p1} : \bar{b}_{p2} \rangle / H[i \vdash R_i][\bar{c} \vdash R_i]}^{\text{RUN}} \\
 \\
 \overline{\langle \bar{b}_r[(i, \bar{c}, \text{done})], \bar{b}_p \rangle / H \rightsquigarrow \langle \bar{b}_r[\epsilon], \bar{b}_p \rangle / H[i \vdash C_i][\bar{c} \vdash C_i]}^{\text{COMPLETE}}
 \end{array}$$

■ **Figure 9** Semantics of the core BoC calculus with histories.

$$\begin{array}{c}
 \overline{i \vdash_b \epsilon}^{\text{WFB-EMPTY}} \quad \overline{i \vdash_b R_i : \bar{S} : (C_i?)}^{\text{WFB-NEMPTY}} \\
 \\
 \overline{\vdash_c \epsilon}^{\text{WFC-EMPTY}} \quad \overline{\vdash_c R_i}^{\text{WFC-SINGLE}} \quad \frac{\vdash_c \bar{E}}{\vdash_c R_i : C_i : \bar{e}}^{\text{WFC-PAIR}}
 \end{array}$$

■ **Figure 10** Well-formedness rules for behaviour and cown histories

4.3 Soundness of Core Calculus According to the Axiomatic Model

In the previous section, we showed that a well-formed history translates to a valid execution in the axiomatic model. In this section we show that the semantics is sound according the axiomatic model by proving that it always produces a well-formed history.

In addition to general well-formedness, the history of an execution is going to match the specific configuration that it was produced together with.

► **Theorem 15** (Preservation of well-formed and matching histories). *Starting from a well-formed and matching history, evaluation produces another well-formed and matching history:*

$$\tau \vdash H \wedge H; \tau \vdash \text{cfg} \wedge \text{cfg}/H \rightsquigarrow \text{cfg}'/H' \implies \exists \tau'. \tau' \vdash H' \wedge H'; \tau' \vdash \text{cfg}'$$

Proof. By cases on the evaluation relation. For each case we select a τ' that is τ extended so that the new event produced by that evaluation step is given a timestamp larger than any previous timestamp. Preservation of each of the two relations can then be proved separately. We refer to the mechanisation for details [7]. ◀

The operational semantics, and its implementations, are sound with respect to the axiomatic model